

X3D Copilot

Technical overview – spec-grounded, self-verifying AI authoring of X3D 4.0 scenes

Alberto Jaspe-Villanueva, KAUST alberto.jaspe@kaust.edu.sa

Prototype: <https://albertojaspe.net/x3d-copilot/> Source (MIT): <https://github.com/ajaspe/x3d-copilot>

To try the AI live: open Settings in the prototype and paste your own Gemini (free tier at aistudio.google.com/apikey) or Claude API key; the key stays in your browser and is sent only to the provider. Everything else works without a key.

Web3D 2026 – AI & Web3D Innovation Competition – category: AI-Driven Content Generation

Overview

X3D Copilot is a browser-only application in which a large language model authors and edits X3D 4.0 scenes under continuous verification. The model never writes to the scene directly: it calls tools that patch or replace the source, and each call returns a validation report produced by the official X3D 4.0 XML Schema, a semantic linter derived from the X3D Unified Object Model, and the X_ITE runtime. A screenshot tool lets the model inspect its own render. Users work alongside the model in a three-pane interface (chat, live X_ITE view, source editor) and can select objects with an X3D-native gizmo; the selection is passed to the model as context.

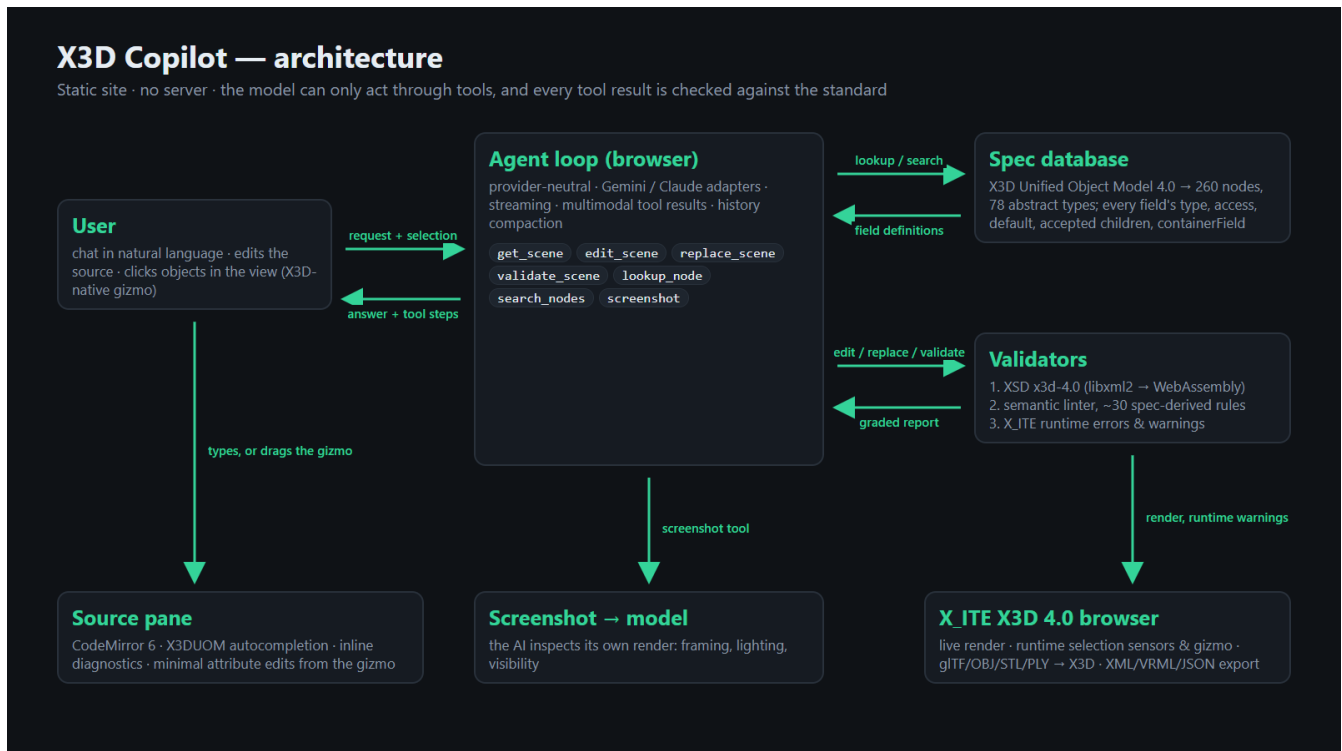


Figure 1: Architecture. Everything runs client-side; the only network traffic is the model API call made with the user's own key.

AI integration

Models. Google Gemini (3.8 Flash by default; 3.1 Pro, 2.5 Pro/Flash selectable) and Anthropic Claude (Opus 5, Sonnet 5, Haiku 4.5). Both are reached through their official JavaScript SDKs directly from the browser; a provider interface (`runTurn`, `listModels`, `describeError`) isolates each SDK in an adapter of about 150 lines. Streaming text, native function calling, multimodal tool results (images returned inside function responses), thinking-level/effort control and prompt caching are used where the provider supports them.

Agent loop. A provider-neutral loop (`src/ai/agent.ts`) sends the system prompt, tool definitions and history; executes returned tool calls locally; appends results; and repeats up to 16 rounds per user turn. Stop reasons (max tokens, refusal, safety) are surfaced to the user. History is compacted after each turn: old screenshots and whole-scene tool inputs are replaced by short placeholders, since the scene is always re-readable via `get_scene`.

Tools (JSON-schema definitions with local executors):

- `get_scene`, `edit_scene` (exact, unique text replacements or anchored insertions; rejected if ambiguous), `replace_scene`, `validate_scene` – each returns the graded issue list (error/warning/hint with line numbers and sources) plus the X_ITE render status;
- `lookup_node`, `search_nodes` – read the ISO object model: fields with type, accessType, default, accepted node types and containerField, component and spec URL; nodes the running X_ITE build does not implement are flagged;
- `screenshot` – returns the current frame as a PNG so the model can check the result visually.

Prompting. The system prompt fixes the workflow (read before edit, prefer patches, fix errors before answering, look up when unsure, screenshot when appearance matters) and a compact X3D 4.0 authoring guide (structure, animation and interaction idioms, field syntax, X3D4 features). A per-message context note carries the user's current selection. The prompt is deliberately short; correctness comes from the tools, not from prompt text.

Verification stack

1. **XSD:** `x3d-4.0.xsd` with the Web3D extension schemas and the W3C XML-Signature schema are vendored and compiled once by `libxml2` running in `WebAssembly` (`libxml2-wasm`); validation of a typical scene takes a few milliseconds.
2. **Semantic linter:** ≈ 30 rules that the schema cannot express, all driven by the object-model database: duplicate DEF, undefined USE, USE of a different node type, ROUTE to unknown node/field, ROUTE from non-output or to non-input fields, ROUTE type mismatch, node in a parent field that does not accept it (e.g. **Material** inside **Shape**), several children in an SFNode field, attribute arity/type/range checks, zero rotation axes, interpolator key/keyValue consistency, empty groups, Shape without geometry, missing Viewpoint, unknown nodes/fields with did-you-mean suggestions, and nodes unsupported by the current X_ITE build.
3. **Runtime:** X_ITE parse errors and console warnings (missing textures, bad URLs, unsupported profiles) are captured during load and reported.

Schema messages that duplicate a linter finding on the same line are demoted to hints so the model is not double-penalised; hints are hidden from the model unless it asks for a full validation.

Web3D frameworks and standards

X3D 4.0 (ISO/IEC 19775-1:2023) and its XML encoding (ISO/IEC 19776-1); X3D Unified Object Model 4.0 (Web3D Consortium), compiled at build time into `src/generated/x3duom.json` (260 concrete nodes, 78 abstract types, 17 statements); X_ITE 16.3 as the rendering and runtime engine (including its glTF/OBJ/STL/PLY/SVG loaders and XML/Classic VRML/JSON encoders); CodeMirror 6 for the editor, with completions generated from the object model.

Direct manipulation as shared context

Clicking geometry selects its innermost **Transform**. The selection machinery is itself X3D: after each load, invisible **TouchSensor** nodes are injected into every Transform via the Scene Access Interface, and the gizmo (move along X/Y/Z with **PlaneSensor**, rotate with **CylinderSensor**, uniform scale) is a group of sensor nodes inserted next to the selected node in its parent's frame. A **ProximitySensor** reports the camera pose so the gizmo keeps a constant screen size and each axis handle uses the drag plane that faces the viewer. Drags are written back to the source as minimal attribute edits through the editor's syntax tree; placing the text cursor on a **Transform** selects it in the view. The current selection (DEF name, line, transform) is appended to the user's message, so deictic requests ("make this twice as big") resolve to an exact node.

Engineering, tests, deployment

TypeScript, Vite; 56 vitest tests (XSD fixtures, one test per linter rule, edit-tool patching, source-map/gizmo math, and a check that every shipped example is lint-clean and schema-valid while the deliberately broken example triggers nine specific rules); CI on every push. The production build is a static site (≈ 3 MB of application code plus the vendored X_ITE assets), hosted on the author's website; no server component or accounts are required beyond the user's own model API key, which is stored only in the browser's `localStorage`.

Limitations and next steps

Selection currently targets **Transform** nodes only; `ProtoInstance` internals are not selectable. The screenshot check is qualitative. Planned: HAnim-aware tools (joint lookup, motion capture import), glTF export via X_ITE, a retrieval tool over the X3D examples archive, and applying the same architecture (spec database + validators as tools + vision check) to glTF and USD.